

От анализа кадров к анализу движения

Переход от изображений к видео означает появление новой оси — времени. Пространственная структура остаётся важной, но ключевым источником информации становится динамика.

Пространственная структура

Фиксированный момент времени,
статичные объекты и границы

Временная динамика

Последовательное изменение,
движение объектов,
коррелированная структура

Потеря информации

Усреднение кадров теряет жесты,
перемещения и межкадровые
зависимости

Видеоряд как тензор: $\mathbf{X} \in \mathbb{R}^{T \times H \times W \times C}$, где T — длина клипа, H, W — пространственные размеры, C — число каналов.

Пространственно-временной сигнал

Видео — это дискретизация трёхмерного сигнала $I(x,y,t)$, зависящего от координат и времени. Движение интерпретируется как изменение интенсивности пикселей.

Временная производная

$$\partial I(x, y, t) / \partial t$$

Отражает скорость изменения яркости и формирует оптические потоки

Реальные искажения

- Шум и сжатие видео
- Изменение освещения
- Дефокус камеры
- Нестационарность сигнала

Признаки возникают не в отдельных кадрах, а в зависимости между ними. Пространство признаков должно включать временную компоненту.

Классификация действий

Задача: предсказать метку действия из множества категорий для видеоклипа фиксированной длины T .

Модель реализует отображение: $\mathbf{f}: \mathbb{R}^{T \times H \times W \times C} \rightarrow \mathbf{C}$, где $\mathbf{C} = \{c_1, \dots, c_K\}$ — множество действий.

Локализация действий

Начало и конец действия не совпадают с границами клипа

Различная длительность

Фиксированная длина клипа — инженерный компромисс

Шум данных

VFR-запись, колебания скорости камеры, артефакты сжатия

Датасеты и метрики

101

UCF101

Действия для базового тестирования

51

HMDB51

Компактный набор действий

700

Kinetics-700

Крупномасштабный датасет для
обучения

Тор-1 точность

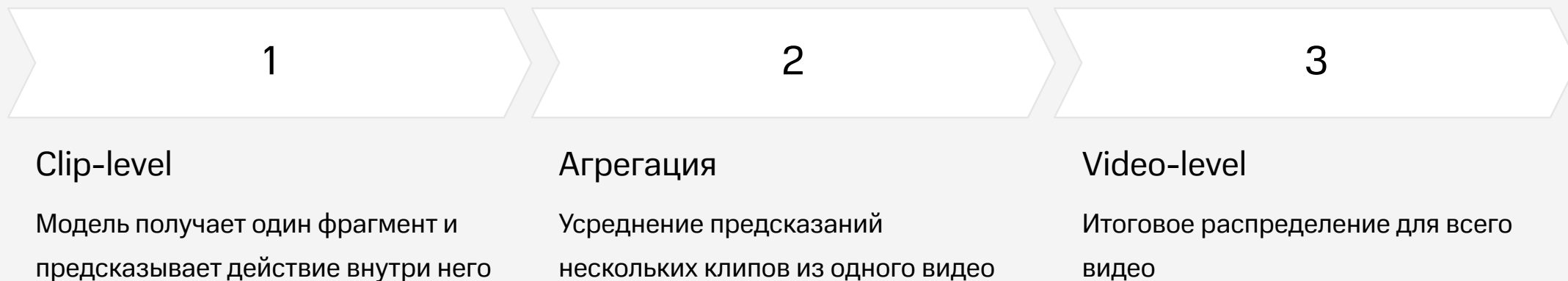
Доля примеров, где действие с максимальной вероятностью
совпадает с истинным

Тор-5 точность

Входит ли истинное действие в пять наиболее вероятных
предсказаний

Clip-level и Video-level предсказания

Клип — фрагмент видео фиксированной длины (8–64 кадра). Действие может быть длиннее клипа или выходить за его границы.



Формула агрегации: $\hat{p}_{\text{video}}(c) = (1/N) \sum \hat{p}(c|X_i)$

Проблема нестационарности

Видеоряд редко стационарен: скорость движения меняется, камера двигается, освещение нестабильно, cadence кадров нарушается.



Изменчивая структура

Зависимость между соседними кадрами не всегда одинакова



Математическое проявление

$I(x,y,t) \neq I(x,y,t+\tau)$ —
статистики зависят от времени



Требования к архитектуре

Устойчивость через
инвариантные фильтры или
механизмы внимания

Пространственно-временные фильтры

Трёхмерные свёртки — естественное расширение двумерных для учёта локальных временных зависимостей.

Формула трёхмерной свёртки:

$$Y(t, x, y) = \sum_{\tau=0}^{k_t-1} \sum_{i=0}^{k_x-1} \sum_{j=0}^{k_y-1} W(\tau, i, j) \cdot X(t - \tau, x - i, y - j)$$

Преимущества

- Выделение паттернов движения
- Учёт соседних кадров
- Чувствительность к динамике

Недостатки

- Высокая вычислительная сложность
- Рост параметров пропорционально kt
- Ограниченный temporal receptive field

Интуиция трёхмерных свёрток

Трёхмерная свёртка одновременно выделяет пространственные границы и динамику. Фильтр реагирует на изменение формы объекта.



Жесты

Характерный временной рисунок махания рукой



Походка

Периодическое движение конечностей



Прыжки

Фазовые переходы: приседание, толчок, полёт

Ограничение: если $kt=3$, фильтр видит только три соседних кадра. Для анализа 16–32 кадров требуется глубокая модель.

Влияние частоты кадров и длины клипа

Структура движения зависит от FPS. Высокий FPS даёт плавное движение, низкий — дискретное.

Дискретизация движения: $\mathbf{v}(\mathbf{t}) \approx [\mathbf{x}(\mathbf{t}+\Delta\mathbf{t}) - \mathbf{x}(\mathbf{t})] / \Delta\mathbf{t}$

Высокий FPS

Много межкадровой информации, плавное движение

Переменный FPS

VFR-запись, требует устойчивых архитектур

1

2

3

Низкий FPS

Рывки, большие смещения объектов

Длина клипа T должна быть согласована с FPS: слишком короткий клип не содержит действие целиком, слишком длинный увеличивает требования к памяти.

Переход от 2D к 3D CNN

Двумерные свёртки теряют ключевую часть информации — структуру изменения. Агрегация признаков через усреднение не различает причинно-следственные связи.



Трёхмерная свёртка: формальное определение

Трёхмерная свёртка сопоставляет движущийся шаблон фрагменту пространственно-временного объёма.

Входной тензор: $\mathbf{X} \in \mathbb{R}^{T \times H \times W \times C_{in}}$

Ядро свёртки: $\mathbf{W} \in \mathbb{R}^{k_t \times k_h \times k_w \times C_{in} \times C_{out}}$

Выходная активация:

$$Y(t, x, y, c) = \sum_{\tau=0}^{k_t-1} \sum_{i=0}^{k_h-1} \sum_{j=0}^{k_w-1} \sum_{m=0}^{C_{in}-1} W(\tau, i, j, m, c) \cdot X(t - \tau, x - i, y - j, m)$$

Ядро охватывает несколько последовательных кадров, активация зависит от согласованной структуры в них.

Архитектура C3D

C3D — одна из первых успешных реализаций 3D CNN для распознавания действий. Использовала однородные ядра $3 \times 3 \times 3$, структура напоминала VGG.

01

Входной клип

16 кадров пропускаются через
последовательность свёрточных слоёв

02

Pooling по времени

Уменьшение пространственных и
временных размеров

03

Иерархия паттернов

От элементарных движений к
семантическим действиям

Достижения: впервые показала, что 3D CNN значительно превосходят 2D CNN с усреднением по времени на UCF101 и HMDB51.

Вычислительная сложность 3D свёртки

Трёхмерная свёртка существенно дороже двумерной. Количество параметров и FLOPs увеличиваются пропорционально kt .

Вычислительная сложность:

$$FLOPs \approx T \cdot H \cdot W \cdot kt \cdot kh \cdot kw \cdot Cin \cdot Cout$$



Увеличение стоимости при ядре $3 \times 3 \times 3$ по сравнению с 3×3



Рост параметров в глубоких сетях

Это критично для обучения на больших датасетах и применения в продакшене. Требуются упрощения через факторизацию.

Receptive Field в 3D CNN

Временной receptive field определяется глубиной сети и размером ядра.

Эффективная длина после L слоёв:

$$R_t = 1 + (k_t - 1) \sum_{\ell=0}^{L-1} s_t \ell$$

Линейный рост

Stride больше единицы

При $k_t=3$ и $s_t=1$ для охвата 32 кадров требуется ~15 слоёв

Быстрый рост, но потеря информации о мелких движениях

Инженерный компромисс: небольшое k_t и осторожное использование temporal pooling.

Проблема вычислительной сложности

Трёхмерные свёртки слишком дороги для реальных задач. Число параметров растёт пропорционально kt , что увеличивает потребление памяти.

Ограничения батча

Тензор видеоклипа плохо укладывается в GPU при больших T

Трудности обучения

Ограниченный размер батча ухудшает стабильность оптимизации

Inference

Требует значительных ресурсов, неприменимо для реального времени

Решение: факторизация 3D-операции на 2D и 1D части для уменьшения параметров при сохранении временной чувствительности.

R(2+1)D: декомпозиция на 2D+1D

R(2+1)D разделяет трёхмерную свёртку на последовательность двумерной пространственной и одномерной временной свёрток.

Факторизация:

$$Conv3D(kt, kh, kw) \approx Conv2D(kh, kw) \circ Conv1D(kt)$$

1

Пространственная свёртка

Выделение форм и границ объектов

2

Временная свёртка

Анализ динамики вдоль временной
оси

3

Нелинейность между ними

Увеличение выразительности модели

Преимущества: уменьшение параметров, увеличение глубины, более устойчивое обучение.

Сравнение C3D и R(2+1)D

R(2+1)D превосходит C3D по точности на тех же датасетах при меньшей вычислительной стоимости.

30%

Снижение FLOPs

R(2+1)D легче при сопоставимой точности

+5%

Прирост точности

На Kinetics по сравнению с C3D

C3D

- Требует значительных ресурсов
- Быстрое насыщение качества
- Ограниченная масштабируемость

R(2+1)D

- Более глубокие сети без роста параметров
- Устойчивое обучение
- Способность к улучшению при увеличении глубины

I3D: Inflated 3D ConvNet

I3D предложила концепцию «надувания» двумерных фильтров в трёхмерные для использования предобученных весов ImageNet.

Формула inflation:

$$W_{3D}(\tau, i, j) = (1/kt) \cdot W_{2D}(i, j)$$

для всех $\tau \in [0, kt-1]$



Предобученные веса

Использование мощных моделей
ImageNet



Стабилизация

Перенос весов стабилизирует
оптимизацию



Ускорение

Быстрая сходимость при обучении

I3D стала первой моделью с высокими результатами на Kinetics-400.

Feature Fusion и Temporal Pooling

Классические 3D CNN используют различные стратегии агрегации временной информации.

Среднее по времени:

$$\bar{F}(x, y, c) = (1/T') \sum_t F(t, x, y, c)$$

	Temporal Pooling Сокращение размерности по времени в глубоких слоях		Инвариантность Устойчивость к малым изменениям продолжительности движения		Потеря структуры Усреднение может терять важную динамику действия
---	---	---	---	---	---

Альтернативы: max-pooling по времени или многоступенчатые схемы для сохранения временной структуры.

Мотивация к гибридным архитектурам

Ограничения классических 3D CNN стимулируют разработку гибридных архитектур.

1

Вычислительная тяжесть

Модели слишком дороги для длинных клипов

2

Медленный рост receptive field

Требуется много слоёв для охвата длинных действий

3

Потеря структуры

Pooling-операции теряют важную динамику

4

Плохая масштабируемость

Неприменимы в сценариях реального времени

Следующий шаг: SlowFast и трансформерные модели с гибким балансом пространственной и временной обработки.

SlowFast: два потока времени

SlowFast рассматривает видео как систему двух разных скоростей времени: медленной (пространственная семантика) и быстрой (динамика).

Slow Pathway

Редкие кадры (f/α), высокоуровневые пространственные признаки, полная ширина каналов

Fast Pathway

Много кадров (f), специализация на динамике, уменьшенная ширина каналов (βC)

Коэффициенты: $\alpha = 4-8$ (замедление), $\beta \ll 1$ (уменьшение каналов)

Идея: пространственная семантика меняется медленно, движение — быстро. Декомпозиция на низкие и высокие частоты.

Математическая модель SlowFast

Slow-путь получает выборку: **$Is(\mathbf{x}, \mathbf{y}, tk)$** , где $tk = k\alpha\Delta t$

Fast-путь получает: **$If(\mathbf{x}, \mathbf{y}, t)$** , где $t = k\Delta t$

Взаимодействие потоков:

$$S'(tk) = g(S(tk), h(F(tk - \delta), \dots, F(tk + \delta)))$$

где h — оператор агрегации быстрых признаков, g — оператор объединения потоков

Медленный поток использует быстрый как источник динамической информации. Многомасштабное моделирование временных зависимостей.

Slow Pathway: пространственные признаки

Медленный путь обрабатывает редкие кадры с полной сетевой шириной, обеспечивая высокую емкость модели.



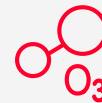
Позы

Распознавание ориентации и
положения объектов



Контекст

Анализ окружения и сцены



Структура

Сложная пространственная
информация

Входной тензор: $\mathbf{X_s} \in \mathbb{R}^{T_s \times H \times W \times C}$, где $T_s = T/\alpha$

Низкая временная частота позволяет использовать полноценные трёхмерные свёртки без переполнения памяти.

Fast Pathway: быстрые изменения

Быстрый путь получает все кадры с частотой f , но пространство каналов уменьшено до βC ($\beta = 1/8$).

Мелкие движения

Восприятие коротких всплесков активности

Высокочастотные сигналы

Все промежуточные состояния действия

Микродвижения

Жесты, направления, резкие фазовые переходы

Fast-путь можно рассматривать как оценку производной видеосигнала — наибольшая чувствительность к малым изменениям.

Критичен для различия действий типа «удар», «бросок», «аплодисменты» с короткими пиками скорости.

Fusion Block: объединение потоков

Слияние выполняется через lateral connections — боковые соединения, передающие признаки fast-пути в slow-путь.

Формула объединения на уровне l :

$$S'l = Sl \oplus \varphi(Fl)$$

где φ — преобразование (усреднение по времени + линейная проекция), $\oplus$ — конкатенация или суммирование

Slow получает динамику

Fast остаётся лёгким

Точность Fast-пути с богатой структурой признаков

Служит «энергетическим» каналом движения

Баланс вычислений между потоками

Fast-путь получает больше кадров, но узкий и дешёвый. Slow-путь широкий, но получает меньше кадров.

FLOPs Fast-пути:

$$FLOPs_F \approx T \cdot HW \cdot k^3 \cdot (\beta C)^2$$

FLOPs Slow-пути:

$$FLOPs_S \approx (T/\alpha) \cdot HW \cdot k^3 \cdot C^2$$



Уменьшение сложности Fast-пути при $\alpha=8$, $\beta=1/8$



Общая сложность по сравнению с однотемповой 3D CNN

Баланс делает SlowFast практичной для промышленных задач.

Устойчивость к FPS и Stride

SlowFast устойчива к вариативности FPS благодаря быстрому потоку, который получает больше кадров.



Высокая устойчивость к реальным видеоданным с переменным FPS и пропусками кадров. Предпочтительный выбор для приложений вне лаборатории.

Реализация SlowFast в PyTorch

Упрощённый пример блока SlowFast:

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class SlowFastBlock(nn.Module):
    def __init__(self, C_slow, C_fast, alpha=8):
        super().__init__()
        self.alpha = alpha
        self.conv_slow = nn.Conv3d(C_slow, C_slow,
                                     kernel_size=3, padding=1)
        self.conv_fast = nn.Conv3d(C_fast, C_fast,
                                     kernel_size=3, padding=1)
        self.fuse = nn.Conv3d(C_fast, C_slow, kernel_size=1)

    def forward(self, x_slow, x_fast):
        x_s = self.conv_slow(x_slow)
        x_f = self.conv_fast(x_fast)
        x_f_ds = F.interpolate(x_f, size=x_s.shape[2:])
        return x_s + self.fuse(x_f_ds)
```

Применение SlowFast

SlowFast показала значительный прирост качества в распознавании действий и видео-детекции.

+8%

Kinetics

Прирост точности по сравнению с 3D CNN

+12%

HMDB51

Улучшение на сложных действиях

Action Recognition

- Устойчивость к шуму
- Глобальная структура
- Кратковременные сигналы

Video Detection

- Бэкбон для двухступенчатых моделей
- ROI pooling по времени
- Границы объектов + движение

Переход к трансформерным моделям

SlowFast наследует ограничения свёрточных моделей: ограниченный temporal receptive field, зависимость от ядра, фиксированная структура соседства.

1

CNN

Локальное ядро, глубокая сеть для длинных зависимостей

2

Attention

Связи между любыми моментами времени без локального ядра

3

Трансформеры

TimeSformer, VideoSwin — гибкое моделирование зависимостей

Внимание не нуждается в локальном ядре, вычисляет связи между любыми моментами времени. Гораздо более гибкий подход.

От CNN к Self-Attention

Переход к трансформерам основан на фундаментальном недостатке CNN — ограниченной видимости. Даже глубокие 3D CNN имеют конечный receptive field.

Self-attention вычисляет взаимодействия между всеми элементами последовательности напрямую, без фиксированного ядра.

Универсальность

Каждый элемент устанавливает
связи со всеми остальными

Дальние зависимости

Долгосрочные взаимосвязи без
роста числа слоёв

Параллельные события

Множество взаимодействующих
объектов

Концептуальный сдвиг: отказ от предположения о локальных паттернах. Весь видеоклип — последовательность взаимодействующих элементов.

Video Vision Transformers

Для применения ViT к видео необходимо преобразовать видеоряд в последовательность патчей.

Последовательность токенов: $\{\mathbf{x}_{t,p}\}$, где $t=1,\dots,T$, $p=1,\dots,N_p$

Проекция патча:

$$z_{t,p} = E \cdot x_{t,p} + \text{epos}(t,p)$$

где E — матрица проекции, $\text{epos}(t,p)$ — позиционный эмбеддинг

Длина последовательности: $T \cdot N_p$

Сложность self-attention: **$O((TN_p)^2)$**

Для крупных клипов требуется факторизация внимания → TimeSformer, VideoSwin

TimeSformer: факторизация внимания

TimeSformer факторизует внимание по двум осям: пространственной и временной.

Уменьшение сложности с $O((TNp)^2)$ до $O(TNp^2 + NpT^2)$

Spatial Attention

$$Attn_{space}(Z_t, :) = softmax(Q_t K_t^\top / \sqrt{D}) V_t$$

Внимание в пределах каждого кадра

Temporal Attention

$$Attn_{time}(Z :, p) = softmax(Q :, p K :, p^\top / \sqrt{D}) V :, p$$

Между патчами с одинаковым индексом в разных кадрах

Интуиция: пространство и время имеют разную структуру. Пространственное внимание — геометрия, временное — динамика.

Формула Self-Attention в видео

Self-attention действует над токенами (t,p). Последовательность: $Z = \{z_1, ..., z_L\}$, где $L = T \cdot Np$

Формула self-attention:

$$Attn(Z) = softmax(QK^T / \sqrt{D})V$$

где $Q = ZWQ$, $K = ZWK$, $V = ZWV$

01	02	03
Проекции	Веса внимания	Взвешенная сумма
Вычисление Query, Key, Value из токенов	Softmax от скалярного произведения Q и K	Агрегация Value по весам внимания

Divided Attention и Patch Embedding

В TimeSformer используется divided attention — разделённое внимание по осям. Производительность зависит от построения patch embeddings.

Patch embedding:

$$x_{t,p} = \text{reshape}(X[t, pP : (p+1)P, :])$$

$$z_{t,p} = E \cdot x_{t,p} + \text{epos}(t, p)$$

Размер патча P

Большой P теряет локальную динамику, малый P увеличивает длину последовательности

Позиционные эмбединги

Кодируют и время, и пространство: абсолютные или относительные

Относительные эмбединги учитывают динамическую природу движения и повышают качество временной обработки.

VideoSwin Transformer

VideoSwin ограничивает attention оконной структурой. Пространство разбивается на окна фиксированного размера, внимание вычисляется только внутри окон.

Attention в окне Ω :

$$Attn_{\Omega}(Z) = softmax(Q_{\Omega}K_{\Omega}^{\top}/\sqrt{D})V_{\Omega}$$



Оконное внимание

Уменьшение вычислительной сложности



Shifted Windows

Сдвиг окон для перекрытия и передачи информации



Иерархическая структура

Признаки на разных уровнях разрешения

VideoSwin масштабируема к высоким разрешениям и стала стандартом 2021–2023 годов.

Temporal Shift и Sliding Attention

Temporal Shift Module (TSM) предлагает частичное перемещение каналов во времени:

$$\tilde{X}(t) = X(t+1) \text{ для } \gamma C \text{ каналов, } X(t-1) \text{ для другой части, } X(t) \text{ для остальных}$$

Смешивание каналов

Чувствительность к временным изменениям без 3D внимания

Sliding Attention

Ограничение внимания соседними кадрами или локальными окнами

Ускорение

Применимость к длинным клипам в продакшене

Пример Self-Attention над клипом

Упрощённый пример self-attention для видеоклипа:

```
import torch
import torch.nn as nn

class VideoSelfAttention(nn.Module):
    def __init__(self, dim, num_heads=8):
        super().__init__()
        self.attn = nn.MultiheadAttention(
            dim, num_heads, batch_first=True
        )

    def forward(self, x):
        # x: [B, T*N_p, D]
        out, weights = self.attn(x, x, x)
        return out, weights
```

Блок получает последовательность токенов после patch embedding и вычисляет весовые связи между всеми элементами.

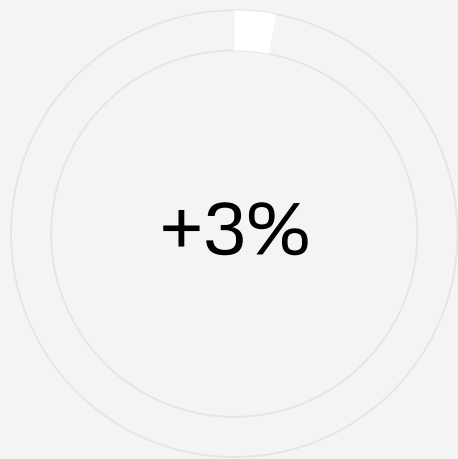
Сравнение 3D CNN и ViT

ViT-модели обеспечивают гибкость и качественное моделирование дальних зависимостей, но требуют контроля вычислительных затрат.

Self-Attention

Сложность: $O(L^2D)$

где $L = T \cdot Np$



Прирост Top-1 точности ViT на Kinetics

3D CNN

Сложность: $O(THW \cdot ktkhkwC^2)$

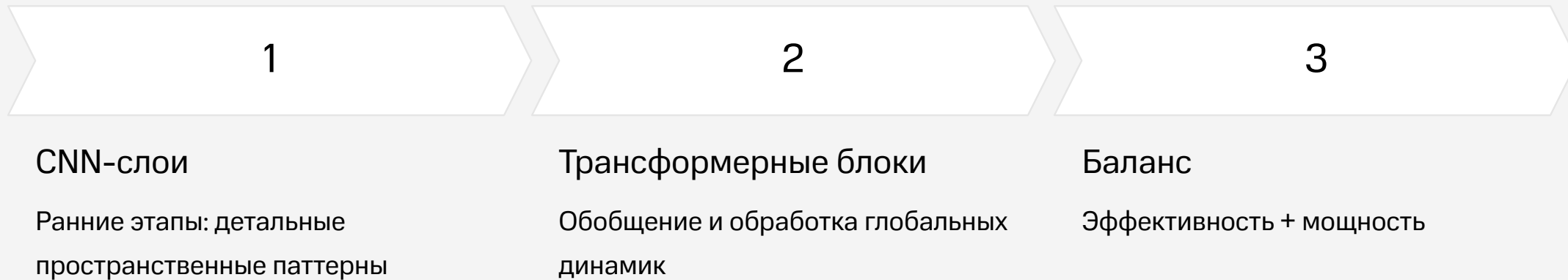


Сравнимая стоимость при оптимизации внимания

ViT превосходит CNN при использовании больших предобученных весов и оптимизации внимания.

Гибридные модели: CNN+Transformer

Многие модели 2023–2025 годов используют гибридные архитектуры: CNN для локальных признаков, трансформеры для глобальных связей.



Гибриды устраняют слабости обеих парадигм: CNN ограничены локальным ядром, ViT — квадратичной сложностью.

Долгосрочные зависимости + устойчивые пространственные признаки + разумная вычислительная стоимость.

Feature Aggregation в видео

Задача: превратить тензор $F \in \mathbb{R}^{T' \times H' \times W' \times C}$ в вектор фиксированной длины для классификации.

Простейший подход — глобальное усреднение:

$$z = (1/T'H'W') \sum_{t,x,y} F(t, x, y, :)$$

Проблема: теряются сведения о фазах движения и ключевых событиях

1

CLS-токен

Центр притяжения информации в ViT

2

Attention Pooling

Модель учится взвешивать регионы

3

Temporal Attention

Self-attention слой агрегирует динамику

Временные эмбеддинги

Позиционные и временные эмбеддинги кодируют структуру последовательности. Без них self-attention не знает порядок кадров.

Абсолютные

$$et = PE(t)$$

$$z'_{t,p} = z_{t,p} + et + ep$$

Жёстко привязаны к длине клипа

Относительные

$$erel(ti - tj)$$

$$A_{ij} = (Q_i K_j^\top + erel(ti - tj)) / \sqrt{D}$$

Устойчивы к изменениям FPS и длины

Относительные кодировки воспринимают движение как относительные фазовые изменения, а не фиксированные позиции.

Temporal Stride и Receptive Field

Temporal stride определяет частоту дискретизации временной оси. Слишком большой stride пропускает важные моменты, слишком малый делает модель тяжёлой.

Временной receptive field в 3D CNN:

$$Rt = 1 + \sum \ell = 1L(kt(\ell) - 1) \prod j < \ell st(j)$$

SlowFast

Быстрый путь: stride=1, медленный: stride= α

Temporal Shifting

Копирование каналов с соседних позиций

1

2

3

ViT

Stride через пропуск кадров или pooling токенов

Нормализация в видео

Проблема: видеотензоры огромны, размер батча крайне мал. BatchNorm становится шумным.

BatchNorm

$$\mu_B = (1/N) \sum x_i$$

Нестабилен при малом батче

LayerNorm

$$\hat{x}_c = (x_c - \mu) / \sigma$$

Не зависит от размера батча

GroupNorm

Нормализация групп каналов,
устойчива

LayerNorm стала стандартом для Video Transformers. Для 3D CNN используют GroupNorm.

DropPath и регуляризация

Видеомодели легко выучивают ложные корреляции. Регуляризаторы предотвращают переобучение.

DropPath

Stochastic depth: случайный
пропуск резидуальных блоков

Attention Dropout

Подавление излишне острых весов
в матрицах внимания

Декорреляция

Контроль временных признаков

DropPath действует на уровне блоков:

$$y = x + f(x) \text{ если активен, } x \text{ если пропущен}$$

Вероятность пропуска увеличивается с глубиной сети.

Предобучение видеомоделей

Современные видеомодели почти никогда не обучаются с нуля. Обучение проходит в два этапа.

01

Предобучение

Большие видеотекстовые наборы: HowTo100M, VideoCLIP, LAION-Video

02

Дообучение

Задачи action recognition: Kinetics-400/600/700

Контрастивный лосс:

$$L_{contrast} = -\log(\exp(\langle v_i, t_i \rangle / \tau) / \sum_j \exp(\langle v_i, t_j \rangle / \tau))$$

VideoCLIP обучает совместные эмбединги видео и текста.

Transfer Learning: 2D → 3D

Перенос пространственных знаний 2D-модели ускоряет обучение видеомоделей.

Механизм inflation:

$$W_{3D}(\tau, i, j) = (1/kt)W_{2D}(i, j)$$



Инвариантность к масштабу

Уже зафиксирована в ImageNet-моделях



Пространственные границы

Локальные узоры и текстуры



Ускорение обучения

Резкое сокращение времени и данных

В трансформерах: $W_{projvideo} = W_{projimage}$, et добавляется отдельно

Clip-level vs Video-level обучение

Модели обучаются на клипах фиксированной длины T, но должны работать на уровне видео.

Clip-level лосс

$$L_{clip} = -\log \hat{p}(c * |X_{clip})$$

Video-level агрегация

$$p_{video}(c) = (1/N) \sum_i \hat{p}(c|X_i)$$

Случайно выбранный фрагмент видео

Усреднение по всем клипам

Совмещённая схема:

$$L = \lambda_1 L_{clip} + \lambda_2 L_{video}$$

Улучшает способность работать с действиями, растянутыми во времени.

Инженерные схемы обучения

Тренировка видеобэкбона — борьба с ограничениями памяти. Видеоклипы занимают много VRAM, attention требует больших временных тензоров.



Mixed Precision (FP16/BF16)

Уменьшение памяти в два раза



Gradient Checkpointing

Повторный расчёт промежуточных активаций



Distributed Data Parallel

Большие batch-размеры на нескольких GPU



Temporal Activation Recomputation

Пересчёт временных блоков

Пример тренировочной функции

```
def train_step(model, optimizer, clips, labels):  
    with torch.cuda.amp.autocast():  
        logits = model(clips)  
        loss = F.cross_entropy(logits, labels)  
  
        optimizer.zero_grad()  
        scaler.scale(loss).backward()  
        scaler.step(optimizer)  
        scaler.update()  
  
    return loss.item()
```

В реальных проектах один шаг обучения может занимать 100–300 мс даже на A100, если клип длиной 32–64 кадра и модель содержит слои attention с большим количеством токенов.

Многомерные видео-задачи

Обученный видеобэкбон становится фундаментом для целого класса задач.



Детекция действий

Локализация действий в пространстве и времени



Temporal Segmentation

Разбиение видео на временные сегменты



Многообъектный трекинг

Отслеживание траекторий объектов



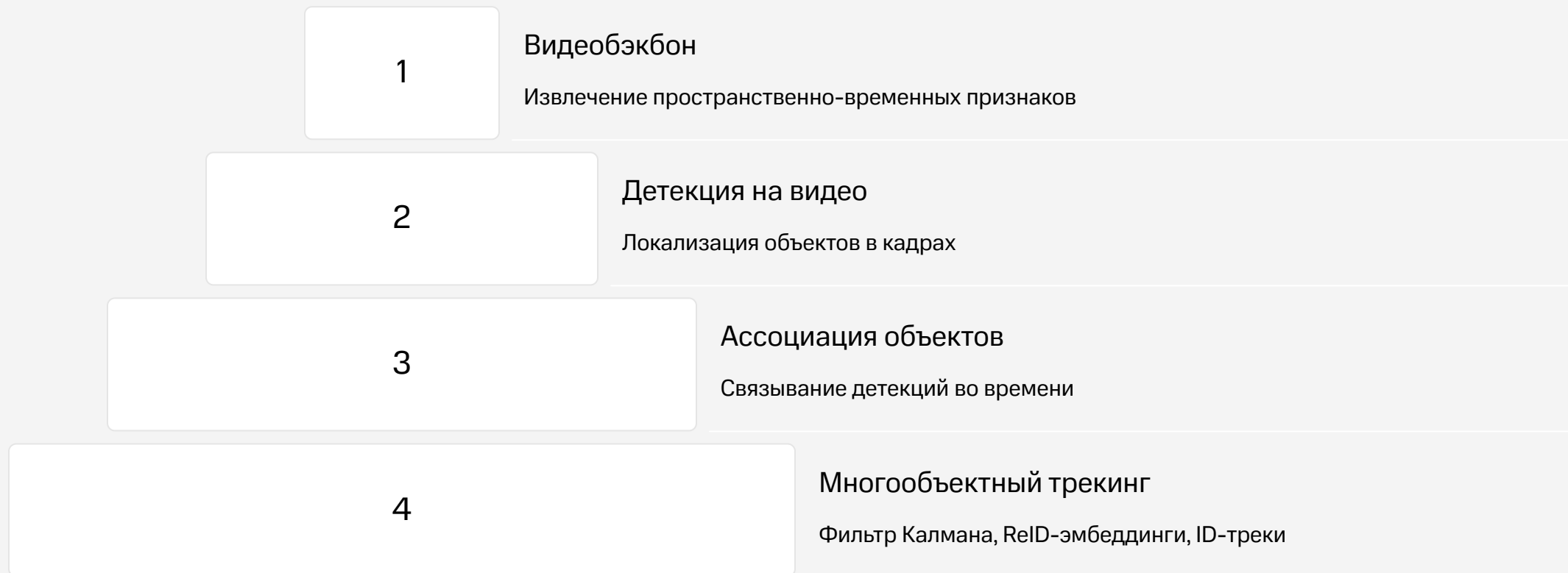
Event Localization

Определение моментов событий

Высококачественный видеобэкбон — ключ к успешной архитектуре следующего уровня.

Переход к детекции и трекингу

Видеобэкбон обеспечивает продвинутые пространственно-временные признаки. Если бэкбон недостаточно чувствителен к динамике, последующие модули не смогут работать эффективно.



Интеллектуальность трекера всегда ограничена качеством его бэкбона. Следующая лекция: детекция на видео и многообъектный трекинг.